Automated Fabric Defect Detection

End-to-end benchmarking report | Roboflow ResNet18 vs custom MobileNetV2 | 30 October 2026

1. Objective and experimental setup

The project develops and compares an automated fabric-image classification workflow using a no-code Roboflow model and a Python/TensorFlow transfer-learning model. The evaluation covers classification performance, training time, inference latency, and visually inspected failures.

2. Dataset preparation

The Roboflow dataset was downloaded as version 2. Its original export contained three folders: Defected, Defect free, and Waste. For the custom model, Waste was merged into the defective class, producing a binary classification task. File counts below reflect the prepared export and may include generated/augmented images rather than unique source photographs.

Split	Defective	Defect free	Total
Train	9,460	1,001	10,461
Validation	903	96	999
Test	450	47	497

The prepared custom dataset is highly imbalanced: approximately 90.4% of the test images are defective. Accordingly, overall accuracy alone is insufficient to assess detection of defect-free fabric.

3. Models and procedure

Roboflow: A hosted ResNet18 classification model was trained on the Roboflow dataset and accessed through a workflow Cloud API. A Colab script sent 50 JPEG images as base64 payloads and saved each filename, predicted label, confidence, latency, and request status to predictions.csv.

Custom model: TensorFlow/Keras MobileNetV2 with ImageNet weights was used as a frozen feature extractor. Input images were resized to 160 x 160 pixels; a global-average-pooling layer, dropout, and sigmoid classification head were added. The model used Adam (learning rate 0.001), binary cross-entropy, batch size 32, and three epochs. Evaluation used the exported test split.

Evaluation caveat: Roboflow model metrics and custom model metrics come from different evaluation configurations, and Roboflow originally retained three classes. They should be interpreted as indicative rather than a strictly controlled head-to-head benchmark.

Performance comparison

4. Reported benchmark metrics

Metric	Roboflow ResNet18	MobileNetV2
Accuracy	Not reported	95.77%
Precision	96.1%	96.0%
Recall	89.8%	96.0%
F1 score	92.5%	96.0%
Training time	Not reported	12.27 min
Model/inference latency	2.3 ms (Cloud API display)	41.23 ms/image (Colab)

The Roboflow dashboard reported 92.5% F1, 96.1% precision, and 89.8% recall. The custom MobileNetV2 achieved 95.77% test accuracy and approximately 96% weighted F1. Roboflow training time and accuracy were not available in the supplied records; they are not estimated here.

5. MobileNetV2 test-set analysis

On 497 test images, the confusion matrix (rows: actual; columns: predicted, ordered defect_free then defective) was [[36, 11], [10, 440]]. This means 11 defect-free samples were incorrectly flagged as defective, while 10 defective samples were missed. The defect-free class had approximately 0.77 F1; the defective class had approximately 0.98 F1.

Actual / predicted	Defect free	Defective
Defect free	36	11
Defective	10	440

6. Cloud inference check on 50 images

The predictions.csv export contains 50 successful API responses (50/50), all with predicted_label = Defected. The mean recorded end-to-end request time was 813.78 ms/image (median 721.07 ms/image; range 620.55–2265.72 ms). These request-level times include network and service overhead and are **not directly comparable** to the Roboflow dashboard's 2.3 ms model latency or the local Colab MobileNetV2 timing.

Because the 50-image CSV does not contain verified ground-truth labels, its classification accuracy cannot be computed. Some filenames suggest mixed image categories, so the all-defective output merits further investigation, but filenames alone do not establish actual labels.

7. Interpretation

MobileNetV2 demonstrates strong overall performance on this particular test export, yet its minority-class performance is notably weaker. The Roboflow ResNet18 dashboard shows good precision and F1, but direct superiority claims are not justified without matched classes, identical held-out images, and consistent latency measurement.

Failure analysis and conclusions

8. Visually inspected failure cases

The verified failure-analysis figure below shows 12 misclassified examples from the MobileNetV2 test set. Eleven are labeled defect-free but predicted defective; one is labeled defective but predicted defect-free. This figure illustrates both false alarms and a missed defect.



Figure 1. Verified misclassified fabric samples, showing ground-truth and predicted classes.

9. Potential causes and improvements

Many samples have subtle, nearly uniform textures, and the displayed failures show that visually similar fabric surfaces can be difficult to distinguish. Potential contributors include subtle defect features, changes in illumination or color, image resizing, and class imbalance. These are plausible hypotheses rather than confirmed causal explanations.

Recommended next steps are to inspect labels for the shown examples, apply class weighting or balanced sampling, evaluate minority-class recall separately, and test higher-resolution crops where defects are small. For a fairer benchmark, retrain or remap both systems to the same binary labels and measure both models on one truly held-out dataset under comparable timing conditions.

10. Conclusion

The pipeline successfully demonstrated hosted Roboflow inference, a reproducible custom MobileNetV2 training workflow, exported CSV predictions, model comparison, and visual failure analysis. The custom model achieved 95.77% test accuracy with a 12.27-minute training run. However, class imbalance and different model evaluation settings limit the strength of any comparative performance claim. The 50 API requests succeeded technically, but their classification correctness remains unverified.

Evidence and reproducibility

Supporting artifacts: benchmark_comparison.csv, predictions.csv, failure_analysis_verified.png, the saved MobileNetV2 .keras model, and the Colab notebook. Metrics are transcribed from the supplied CSV files and Colab/Roboflow screenshots. No unreported training time or accuracy has been fabricated.